# Testing multilingual layouts when pairwise coverage is not enough

## A browser feasibility study and an exact analysis of a 12-test budget

**Bekasyl Zhaksylyk · Project lead · 10 November 2026 · Technical report v0.1**

**Status.** Public development report. No peer review, WCAG conformance judgment, or real-application recall estimate is claimed.

### Abstract

A test suite can cover every pair of configuration values and still miss a failure that depends on a third value. In multilingual interfaces, this matters because translation, available width, and user text spacing all affect the same rendered line. We study a 3-locale × 4-width × 2-spacing grid from two sides: instrument feasibility in the open-source IT-Tools application and exact discovery probabilities for a 12-test budget. At this budget there are 906 minimum pairwise suites, of which 453 include a fixed reference. A checkerboard covers all 26 pair assignments but executes only 12 of 24 possible singleton triple triggers. In an standalone browser fixture, the German label in a 320 px viewport clips by 19.48 CSS px only with spacing enabled. The checkerboard misses that cell despite full pair coverage. In IT-Tools, a calibrated line-advance probe measures 17 px before and 21 px after the spacing intervention in the original EN/DE diagnostic; the search task shows how clipping can be recoverable. These observations support a reproducible instrument and an explicit limit on pairwise selection. They do not estimate how often real defects require three factors.

### What the report contributes

1. An exact, inspectable comparison of minimum pairwise and reference-anchored random selection, independent of a sample of random seeds.

2. A rendered counterexample with a wrapping negative control and measured task outcomes. It connects the mathematical missing cell to actual CSS behavior.

3. A field diagnostic that distinguishes line advance, geometry warnings, recoverable clipping and task completion. The public artifact keeps those outcomes separate.

### Why this matters in practice

A team deciding which localized build to check with a small budget needs more than a coverage percentage. It needs to know which failure mechanisms are left untested, whether an automatic warning corresponds to user-visible loss, and which claims can survive a rerun. We make those questions explicit enough for a reviewer to challenge each one from the artifact.

### Contribution statement

Bekasyl Zhaksylyk directs the project and its academic goals. Codex assisted with implementation, execution, analysis and drafting. Human validation and peer review remain open. No university affiliation or endorsement of this report is implied.

# 1. Research questions and prior work

**RQ1 — Design.** At a matched budget of 12, what discovery guarantees and blind spots follow from the 3×4×2 configuration structure? We compute this exactly for conjunctive trigger supports.

**RQ2 — Measurement.** Can one pinned, localized application be exercised repeatedly while keeping rendered geometry distinct from a defined functional action? We use token output and keyboard search. A label can extend beyond a viewport before scrolling and still be operable. Conversely, a click can succeed while its explanatory label is visually clipped.

**RQ3 — Mechanism.** Can an ordinary layout rule produce a three-factor singleton that escapes a valid pairwise suite? We construct the geometry rather than hard-code a result for a named configuration, then compare measured overflow and click outcomes against prespecified controls.

## Where this sits in the literature

Kuhn, Kacker and Lei [1] describe practical interaction testing. Coverage of value pairs is a property of the selected inputs. Inferring defect discovery additionally requires a trigger model and an oracle. Kuhn et al. [2] studied interaction sizes in prior defect corpora. Those corpora do not supply a prevalence estimate for translated layouts under user spacing.

GWALI [3] studies presentation differences that arise under translation. Mahajan et al. [4] address repair of internationalization presentation failures. ReDeCheck [5] checks consistency across responsive layouts. ReFLAIR [6] examines loss under responsive reflow, and Ma11y [7] uses accessibility mutation to evaluate test tools. The current work asks a narrower question: what can a particular finite test budget guarantee when width, locale and spacing vary together, and how should a small field probe distinguish rendered change from an impaired task?

## Operational definition

A configuration is one triple of input settings. A state is a task-specific starting condition, such as an opened command palette. A *trigger support* is the set of configurations in which a specified outcome occurs. A *family* in the constructed fixture is one deliberately seeded mechanism. In later application research a family will require a documented root cause, not just a repeated detector warning.

A combined-only family is absent at the reference and in all one-factor alternatives and present at a configuration differing on at least two axes. This is a statement about the tested grid. It neither proves that every changed axis is causally necessary nor implies generalization to other widths, fonts or languages.

# 2. Browser method and measurement oracle

## Factor levels and setting

The target grid uses EN, DE and FR, viewport widths 1280, 768, 375 and 320 CSS px, and native versus overridden spacing. Height is 900 CSS px and device-pixel ratio is 1. For the field study we use a locally served checkout of IT-Tools at commit d505845f918e946ec300af7b36efc107e2f66e9e [8]. The browser uses an isolated profile, light color scheme, reduced motion and blocked off-origin requests. A locale marker is recorded along with each task. A declared locale is not a claim that every visible string is translated.

## Intervention

The CSS intervention is based on the four minima discussed in WCAG 2.2 SC 1.4.12 [9]: line height 1.5 times font size, paragraph following space twice font size, letter spacing 0.12 em and word spacing 0.16 em. The inherited application script reads baseline styles before setting values and preserves larger comparable numeric baselines. When the baseline is non-numeric, an applied target and verified non-decrease are different observations. The instrument reports unresolved comparisons; it does not interpret them as pass or fail. A change in viewport width is not equivalent to a zoom test.

## Multiline token field

An isolated probe inserts controlled text with 1, 5 and 9 forced lines and estimates line advance from two differences in scroll height, divided by four. The probe neutralizes box constraints and uses short strings so wrap does not drive the slope. We use two glyph strings; require agreement between the two intervals and original and computed-style clone; and calibrate against 14, 21 and 28 px controls with 0.25 CSS px tolerance. The original value, style and scrolling attributes are restored. This measures a controlled advance for the observed typography; it does not reconstruct the rendering of arbitrary mixed-font text.

## Single-line search and optional controls

We record a fixed long synthetic string, visible geometry, selected characters at both ends after Home/End, and navigation after entering the token query. Single-line box height is an instrument reading, not a multiline advance. In the extended run each token switch is also measured before scrolling. The fraction of its rectangle inside the viewport and an element-from-point check at its center characterize initial geometry. A center outside the viewport is a reachability candidate. It is not a failed user task without testing the relevant navigation or scrolling.

## Evidence levels

Record intervention validity separately from candidate geometry. Then test a concrete user action. In a future study a human review will distinguish purposeful truncation, available recovery, and actual loss. The current evidence stops at the observed machine checks and labeled synthetic mechanisms.

# 3. Historical data: what the instrument flags

The October feasibility matrix captured three IT-Tools states in two locales with four widths and two spacing regimes (48 configurations). All 48 captures are retained. Thirty-two satisfy the capture and resolved-intervention checks; 16 have unresolved spacing comparisons. The 16 are all intervention rows in the palette and token states. Removing them would preferentially remove a treatment and particular states.

| October state | Geometry candidates | Meaning |
|---|---|---|
| Home | 224 | Repeated text-element flags across configuration rows |
| Token | 94 | Repeated flags, including viewport boundaries |
| Command palette | 0 | No candidate from this detector; no proof of usability |
| Total | 318 | Screening output in 32 of 48 configurations |

Native and override each have 16 signal-positive configurations out of 24. Their warning counts are 144 and 174 respectively. Widths 320, 375, 768 and 1280 have 40, 40, 105 and 133 candidates respectively. In particular, a naive count would suggest that the widest screen has the most problems. That inference is unsupported: the visible elements, viewport boundary conditions and duplication can differ between rows. The artifact retains per-configuration counts to show why a pooled warning count is not a denominator for recall.

### Original measurement diagnostic, 9 November

The controlled token probe yielded 17 px native and 21 px with spacing in all eight EN/DE × width pairs. The difference is 4 CSS px, approximately 23.5% relative to the native controlled estimate. Both languages and all four widths gave the same numbers on this platform; these are repeated readings within one application, not eight independent sites.

The search area height changed from 21 to 27 px. The long string could extend beyond the right edge at 320 px after the intervention, but the follow-up reached both ends with Home/End and selected the expected endpoint characters. All 16 native/spacing query-navigation checks passed in each of the diagnostic and follow-up runs. This supports the narrower claim that clipping of that particular input did not block the tested search task.

### Historical fixture

An earlier EN/RU/KK fixture with three widths produced 18 configurations, 108 widget measurements and three seeded clipping mechanisms in each of two same-machine runs. All three were found by the reported three-test prefixes. Those fixtures did not express a combined-only trigger. They checked collection, but could not distinguish strategies on the proposed three-factor mechanism. The constructed control in Section 6 is a separate experiment on the target grid.

# 4. Extended IT-Tools replay

On 10 November, the token-only replay completed **12 locale–width pairs / 24 phase observations**, including the four French width settings. All 24 probes met the calibration, slope agreement and value-restoration conditions; all 24 had a stable geometry snapshot. Every native estimate was 17 CSS px and every intervention estimate was 21 CSS px. The four visible token-option labels were recorded for each locale, so the French condition was checked against rendered translation, not inferred from the browser locale alone.

| Readout | Native | Spacing override |
|---|---|---|
| Calibrated token advance observations | 12/12 | 12/12 |
| Estimated token line advance | 17 px | 21 px |
| At 320 px: switch centers not hit at initial viewport | 6/12 | 6/12 |
| At 375–1280 px: switch centers not hit at initial viewport | 0/36 | 0/36 |

The six center misses per regime correspond to partially visible switch rectangles. The result suggests a width-related mechanism at 320 px in this state, not a spacing-only signal. We did not test operability after scroll or keyboard focus for these switches. The repeated counts therefore describe initial hit testing, not six defects.

**Run disposition.** An initial attempt stopped before collection because the copied dependency tree was incomplete. The next attempt timed out while filling the search input after the endpoint selection sequence. We report that as an unresolved replay failure and analyze the successful token-only rerun separately. The EN/DE search findings in Section 3 refer to the previous completed run; no French search result is asserted. The successful rerun used Node 25.4.0, Chrome 156.0.8078.80 and Playwright 1.64.0-alpha-1790635538000 on macOS ARM64.

## Interpreting the field evidence

The local replay was chosen to tighten instrumentation before any application-level study. It checks that locale selection, the intervention, the token-field probe and search transitions can be recorded under a wider set of inputs. The French addition provides one extra translation and keeps the application, component implementation, environment and date close to the original diagnostic. It does not solve independence across projects, language scripts or rendering engines.

A viewport intersection and an active click target describe different things. A partly visible control can be selected via keyboard or after scrolling; a geometry metric alone cannot decide whether it is discoverable. Conversely, a successfully dispatched action does not establish that a sighted user can identify the label. The report deliberately avoids using either measurement as a substitute for the other.

# 5. Exact analysis at a matched budget

There are 24 cells. Covering all locale–width pairs needs at least 12 tests because each test covers exactly one of the 12 pairs. At equality, each locale–width cell contains exactly one of the two spacing levels. Hence a minimum pairwise design is a 3-by-4 binary matrix whose rows and columns each contain both bits: rows cover locale–spacing and columns cover width–spacing. This is necessary and sufficient.

Enumerating all 4,096 bit matrices yields **906** valid minimum suites. Exactly **453** include the EN/1280/native reference, by complement symmetry. One reference-anchored design sets spacing to (locale index + width index) mod 2. Its 12 cells cover all 26 value-pair assignments. The script validates coverage rather than assuming the construction.

## Exact random baseline

A reference-anchored random suite contains the reference and 11 of the remaining 23 cells, sampled uniformly without replacement. There are C(23,11) = 1,352,078 possible suites. If a trigger support contains the reference, discovery is certain. Otherwise, for support size m, the probability is:

**P(hit) = 1 − C(23 − m, 11) / C(23, 11).**

This result follows by counting suites with all 11 cells outside the support. Uniform sampling from the 453 valid anchored pairwise matrices is a separate strategy. A deterministic suite is a third strategy, with a yes/no result for any specified support.

| Trigger order | Assignments | Checkerboard finds | Anchored random mean |
|---|---|---|---|
| One factor | 9 | 9 | 99.67% |
| Two factors | 26 | 26 | 84.54% |
| Three factors | 24 | 12 | 50.00% |

The final column weights assignments equally within the row. It is not a defect distribution. For a specific nonreference singleton the random probability is 11/23, about 47.83%; the 50% mean averages in the guaranteed reference. For random anchored pairwise suites the probabilities for nonreference singletons range from zero to two thirds. The opposite spacing at the reference locale and width is impossible under this minimum construction: the reference already occupies that cell.

## Practical implication

Coverage by the two baselines differs before any browser code runs. Pairwise guarantees discovery of simple supported pair triggers *when the oracle recognizes the outcome*; it offers no guarantee on a singleton triple. A team can reserve some runs for likely triple triggers, or sample alternative pairwise matrices. Neither choice should be sold as better in real applications without a measured trigger distribution and a validated oracle.

# 6. A browser counterexample with controls

The standalone fixture has four buttons. Text lengths vary with locale. A responsive rule constrains the slot at 320 px. A separate spacing rule widens the rendered text. These properties can interact without a script checking whether the locale, viewport and spacing match a magic triple. The test driver independently checks the Range edge against the visible slot and records text scroll dimensions; it also clicks each button and observes a counter.

The positive controls represent a fixed-width locale-only mechanism, a locale/viewport mechanism and the three-factor mechanism. The negative control uses wrapping to preserve the same kind of text at narrow width. Expected signatures are specified in the driver before collection. The clipping check uses an explicit 0.5 CSS px threshold.

| Constructed mechanism | Full-grid clipping cells | Checkerboard clipping cells |
|---|---|---|
| Locale only | 8 | 4 |
| Locale and viewport | 2 | 1 |
| Locale, viewport and spacing | 1 | 0 |
| Wrapping negative control | 0 | 0 |

Chrome 156.0.8078.80 exercised all 24 configurations. All 96 geometry assertions matched their expected signatures, and all 96 clicks passed. The lone triple-only clipping case is **DE / 320 px / spacing**. Its text extends 19.484375 CSS px beyond the visible slot; scroll width is 221 px and client width is 202 px. No other cell clips that label. The comparator screenshots and geometry readings are preserved.

| Locale / width | 1280 | 768 | 375 | 320 |
|---|---|---|---|---|
| EN | Native | Spacing | Native | Spacing |
| DE | Spacing | Native | Spacing | Native * |
| FR | Native | Spacing | Native | Spacing |

* The selected DE / 320 px cell is native. The measured clipping occurs in its unselected spacing counterpart.

### Why a valid suite misses it

Use the declared ordering EN, DE, FR and 1280, 768, 375, 320. For DE/320 the checkerboard selects native spacing. The selected cell covers the required locale–width pair, and the other selected cells cover DE–spacing and 320–spacing separately. The full combination that causes the clip is absent. Two of the three seeded clipping families are found by this suite. Some other minimum pairwise suites do find the singleton: 212 of the 453 anchored designs, or 46.80%, include this cell. Uniform reference-anchored random finds it with probability 11/23.

### What the counterexample proves

There exists a straightforward rendered configuration interaction missed by one valid minimum pairwise design. The observed click is still successful. This is evidence of clipping of a synthetic label, not evidence that a real user would fail to complete an IT-Tools task. The fixture is a constructive witness and a calibration input, not an independent sample of the web.

# 7. Limitations and next study

**Construct validity.** The oracle distinguishes geometric overflow, text advance and specified task behavior. It cannot determine user intent or the full accessibility impact. A language may render a longer label without affecting the task; a button may be clickable yet difficult to identify. Independent adjudication should check the consequence and preserve disagreements.

**Internal validity.** Browser rendering depends on fonts, locale fallback, system settings and paint timing. The extended matrix is an exploratory replay on one pinned application. Before the main study, record actual font resolution and hold browser profiles, input actions and version locks constant. Randomize run order where state or warm-up effects are plausible; log any failed session and preserve its missingness category.

**External validity.** One app and one Chrome engine do not support a population claim. The three test languages use Latin scripts. The older synthetic RU/KK fixture does not remedy the shortage of real applications or rendering environments. The seeded controls purposely expose specific mechanisms; their frequencies have no relationship to prevalence.

**Conclusion validity.** Counts are repeated measurements within states and projects. They should not be treated as independent observations in a significance test. Exact probabilities in Section 5 are conditional on the specified support and design distribution. They quantify selection uncertainty, not uncertainty in defect labeling. No raw candidate count is converted into real-defect precision, recall or cost savings.

### Proposed confirmatory phase

1. Freeze inclusion rules and a candidate register before inspecting failures. Screen diverse independent applications and retain exclusions with reasons. Record actual locale coverage, licensing, dependency locks, fonts, supported interactions and failures to install.

2. Record a full 24-cell grid for predefined states. Group validated failures by root cause. Review detector negatives sampled by project and state; keep unknowns and explicit intervention failures rather than forcing pass/fail.

3. Use independent human reviewers for a stratified subset, blind to the strategy membership. Define expected tasks, success, recoverability and alternatives in a label guide. If independent reviewers cannot be obtained, report feasibility without claiming real-defect recall.

4. Compare anchored random and pairwise at 12 executed configurations. Include OFAT at its actual seven configurations and the reference at one. Show project-level distributions, coverage and elapsed measurement/review costs. Test sensitivity to unknown labels and alternative matrix orientations.

The strongest result available today is a boundary on a coverage guarantee supported by a reproducible browser counterexample. Whether the boundary matters often enough to alter a real testing budget is the next empirical question.

# 8. Replication and references

The archive includes the Python enumeration, tabular evidence, HTML fixture, Playwright driver and the IT-Tools collector. Use the release README for commands, data-field descriptions, commit information and limitations. The public archive excludes private correspondence, contact details, local browser profiles and application credentials.

**Quick check.** Recompute the exact results from 4,096 binary matrices with Python 3 and inspect the archived 24 synthetic rows. Then run the fixture in a clean Chromium profile. Compare the clipping signature rather than requiring pixel-identical images on different operating systems.

**Field replay.** Prepare the pinned IT-Tools checkout using its own dependency lock. Run the collector against the local server. Keep any failed attempt and name the rerun separately. Review each locale marker, calibrated probe, route and switch reading before aggregating. The release summarizes browser outputs rather than redistributing a checkout of IT-Tools.

## References


[1] D. R. Kuhn, R. N. Kacker, and Y. Lei. *Practical Combinatorial Testing*. NIST SP 800-142, 2010. https://doi.org/10.6028/NIST.SP.800-142

[2] D. R. Kuhn, D. R. Wallace, and A. M. Gallo. “Software fault interactions and implications for software testing.” IEEE Transactions on Software Engineering, 2004. https://doi.org/10.1109/TSE.2004.24

[3] A. Alameer, S. Mahajan, and W. G. J. Halfond. “Detecting and localizing internationalization presentation failures in web applications.” ICST, 2016. https://viterbi-web.usc.edu/~halfond/papers/alameer16icst.pdf

[4] S. Mahajan, A. Alameer, P. McMinn, and W. G. J. Halfond. “Effective automated repair of internationalization presentation failures in web applications using style similarity clustering and search-based techniques.” Software Testing, Verification and Reliability, 2021. https://doi.org/10.1002/stvr.1746

[5] T. A. Walsh, G. M. Kapfhammer, and P. McMinn. “Automated layout failure detection for responsive web pages without an explicit oracle.” ISSTA, 2017. https://doi.org/10.1145/3092703.3092712

[6] Y. He et al. “ReFLAIR: Detecting responsive layout reflow issues using multimodal generative AI.” FSE, 2026. https://seal.ics.uci.edu/projects/ReFLAIR/

[7] R. Tafreshipour et al. “Ma11y: A mutation framework for web accessibility testing.” ISSTA, 2024. https://ics.uci.edu/~seal/publications/2024_ISSTA.pdf

[8] IT-Tools project. Source at d505845f918e946ec300af7b36efc107e2f66e9e. https://github.com/CorentinTh/it-tools/tree/d505845f918e946ec300af7b36efc107e2f66e9e

[9] W3C. “Understanding Success Criterion 1.4.12: Text Spacing.” WCAG 2.2 Understanding Documents. https://www.w3.org/WAI/WCAG22/Understanding/text-spacing.html